waph-nagpalvv

WAPH-Web Application Programming and Hacking

Lab 2 Report

Instructor: Dr. Phu Phung

Student

Name: Viplav Nagpal

Email: nagpalvv@mail.uc.edu (viplavnagpal1704@gmail.com)

Short-bio: I love programming and building stuff. I took this course to learn how to build stuff and add security for the stuff I build, including websites, machine learning models and so much more.

Figure 1: A personal headshot of the student, Viplav Nagpal.

Lab 2 - Front-end Web Development

The lab's overview:

Through this lab, I gained a comprehensive understanding of how the three core pillars of the web—HTML, CSS, and JavaScript—interact to create dynamic,

user-friendly applications. I learned how to structure semantic webpages using HTML and style them cleanly using external and internal CSS layouts.

Crucially, I developed a strong grasp of asynchronous programming and the Document Object Model (DOM). By implementing vanilla Ajax and the jQuery library, I learned how to seamlessly send and receive data from a server in the background, updating specific parts of a webpage without interrupting the user experience with a hard reload. Furthermore, integrating the Joke API and Agify API taught me how to parse complex JSON payloads and handle cross-origin resource sharing (CORS) security policies using modern `fetch()` protocols. Overall, this lab successfully bridged the gap between static web design and interactive, API-driven front-end development.

Link to this lab report: [https://github.com/nagpalvv/waph-nagpalvv/tree/main/labs/lab2]

Task 1: Basic HTML with forms, and JavaScript

a. HTML

We developed a foundational HTML file named `waph-nagpalvv.html`. This page includes structural header tags, an image of my headshot, and basic HTML forms configured to send data via HTTP GET and POST requests.

b. Simple JavaScript

We implemented various JavaScript techniques to make the webpage interactive:

- **Inline JavaScript:** We added an `onclick` event to a `<div>` to display the current date and time, and an `onkeypress` event to the form input fields to log keystrokes to the console.
- **Email JS:** We separated the logic to toggle the visibility of an email address into an external file (`email.js`) to protect it from basic scraping.
- **Digital and Analog Clocks:** We used a `<script>` tag within the HTML to create a `setInterval` function that dynamically updates a digital clock every 500 milliseconds. Furthermore, we utilized an external JavaScript file to draw a functioning analog clock on a `<canvas>`.

Task 2: Ajax, CSS, jQuery, and Web API integration

a. Ajax

We added a new input field and a button to handle asynchronous requests. Using vanilla JavaScript and the `XMLHttpRequest` object, we constructed an `xhttp.open("GET", ...)` request that grabs the user's input and sends it to our local `echo.php` application without reloading the webpage.

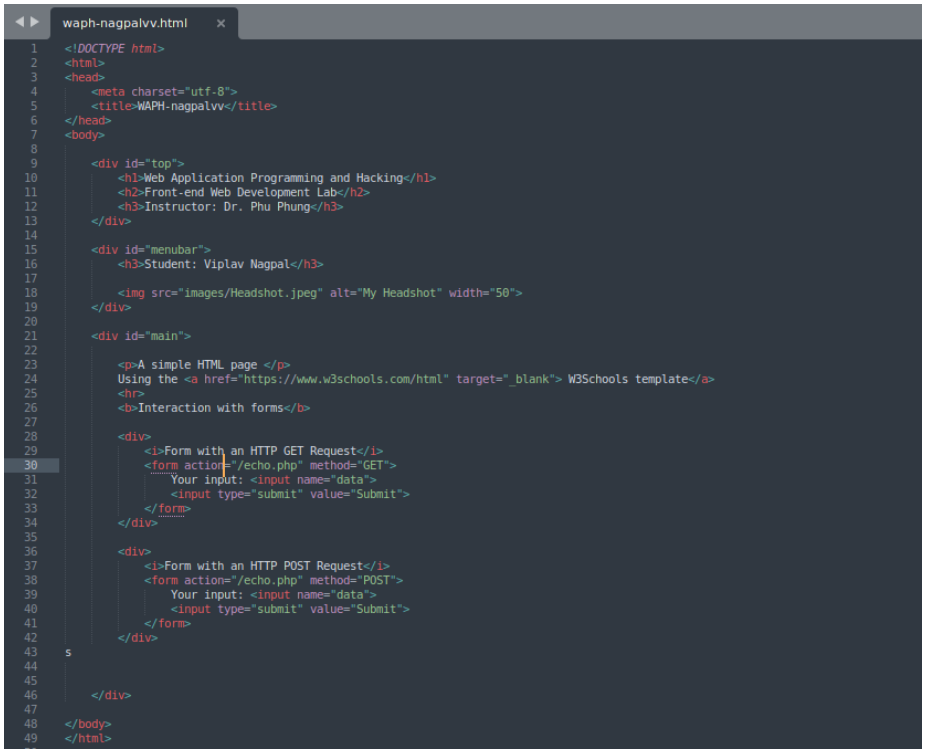

Figure 2: Sublime Text editor showing the foundational HTML skeleton, including header tags and form inputs.

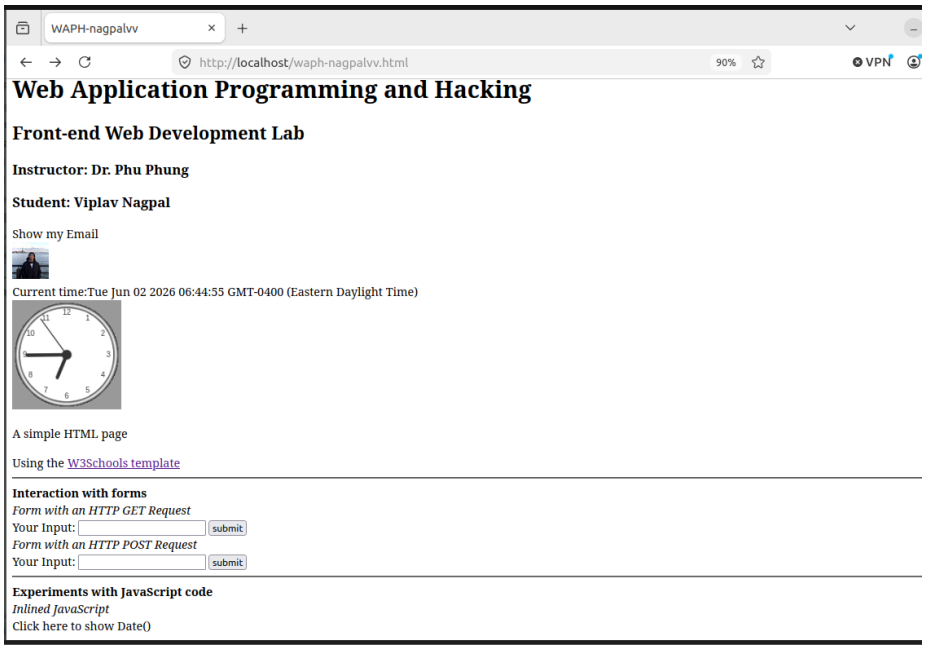

Figure 3: The initial browser rendering of the HTML page, displaying the headshot, headings, and HTTP GET/POST forms.

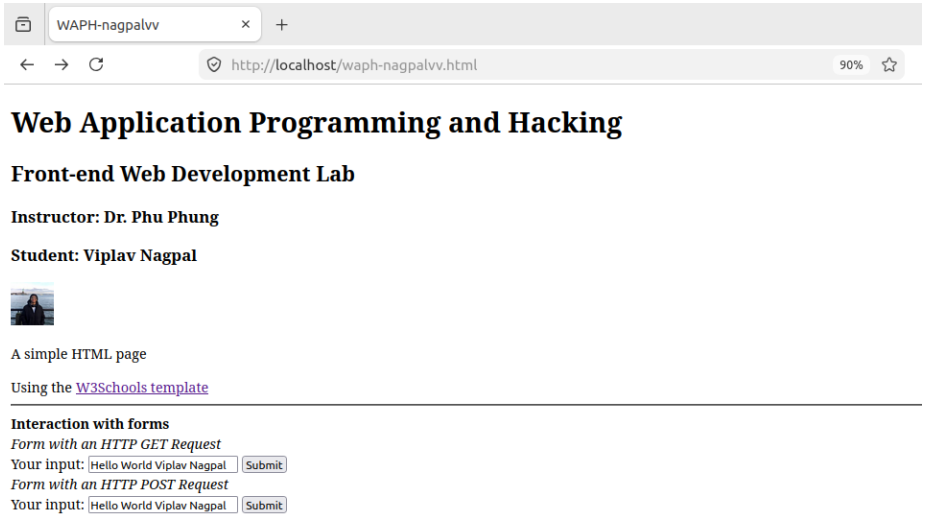

Figure 4: Further progression of the HTML layout before styling is applied.

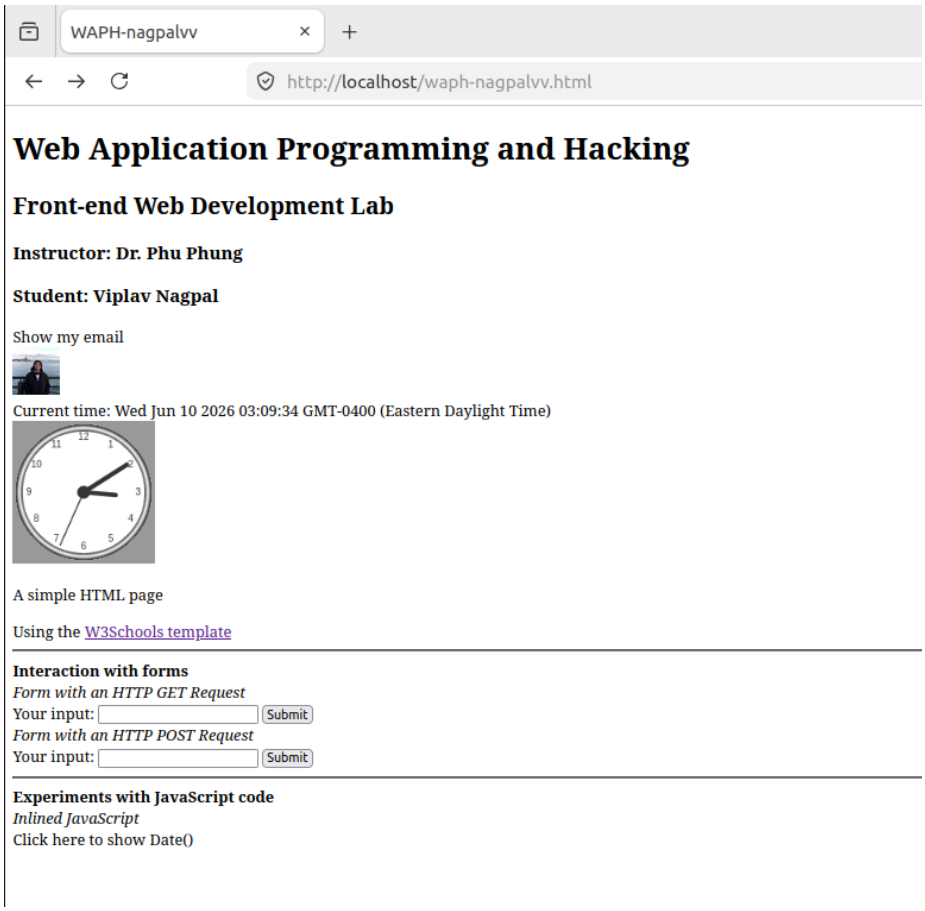

Figure 5: The completed basic layout of Task 1 showing all integrated elements.

```
<div id="main">

    <p>A simple HTML page </p>
    Using the <a href="https://www.w3schools.com/html" target="_blank"> W3Schools template</a>
    <hr>
    <b>Interaction with forms</b>

    <div>
        <i>Form with an HTTP GET Request</i>
        <form action="/echo.php" method="GET">
            Your input: <input name="data" onkeypress='console.log("You have pressed a key")'>
            <input type="submit" value="Submit">
        </form>
    </div>

    <div>
        <i>Form with an HTTP POST Request</i>
        <form action="/echo.php" method="POST">
            Your input: <input name="data" onkeypress="console.log('You have pressed a key')">
            <input type="submit" value="Submit">
        </form>
    </div>
    <hr>
    <b>Experiments with JavaScript code</b><br>
    <i>Inlined JavaScript</i>

    <div id="date" onclick="document.getElementById('date').innerHTML = Date()">Click here to show Date()
    </div>
```

Figure 6: Code snippet demonstrating inline JavaScript for handling click events to show the date.

```
<div id="main">

    <p>A simple HTML page </p>
    Using the <a href="https://www.w3schools.com/html" target="_blank"> W3Schools template</a>
    <hr>
    <b>Interaction with forms</b>

    <div>
        <i>Form with an HTTP GET Request</i>
        <form action="/echo.php" method="GET">
            Your input: <input name="data" onkeypress='console.log("You have pressed a key")'>
            <input type="submit" value="Submit">
        </form>
    </div>

    <div>
        <i>Form with an HTTP POST Request</i>
        <form action="/echo.php" method="POST">
            Your input: <input name="data" onkeypress="console.log('You have pressed a key')">
            <input type="submit" value="Submit">
        </form>
    </div>
    <hr>
    <b>Experiments with JavaScript code</b><br>
    <i>Inlined JavaScript</i>

    <div id="date" onclick="document.getElementById('date').innerHTML = Date()">Click here to show Date()
    </div>
```

Figure 7: Code snippet showing the onkeypress event listener added directly to the form input fields.

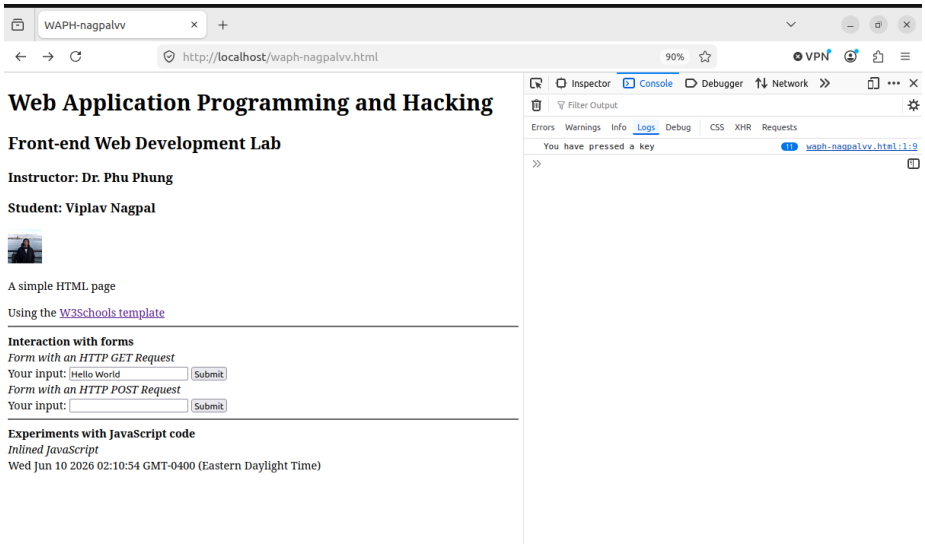

Figure 8: Browser output successfully displaying the current date and time when triggered by the user's click.

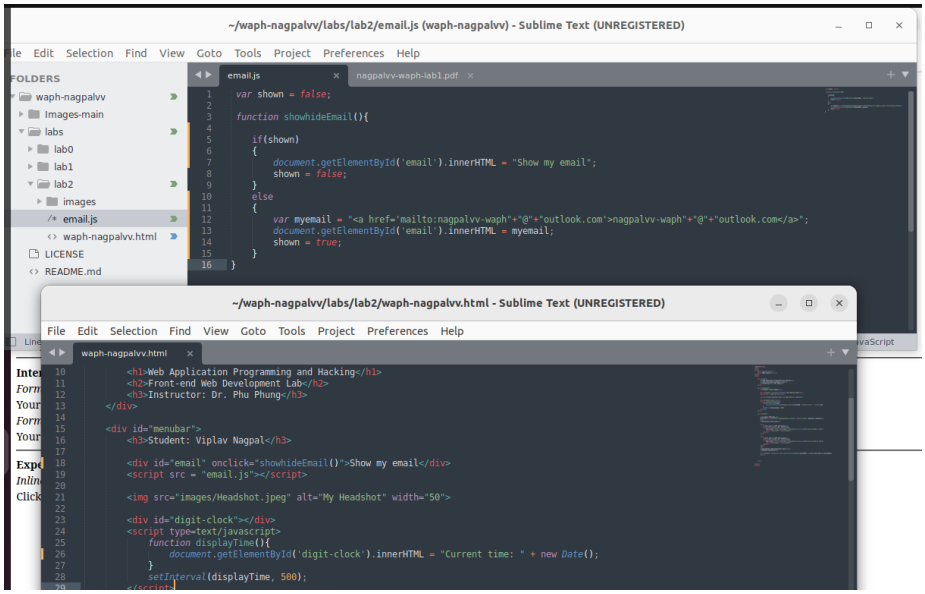

Figure 9: External JavaScript logic used to safely toggle the visibility of the user's email address.

```
14
15      <div id="menubar">
16          <h3>Student: Viplav Nagpal</h3>
17
18          <img src="images/Headshot.jpeg" alt="My Headshot" width="50">
19
20          <div id="digit-clock"></div>
21          <script type=text/javascript>
22              function displayTime(){
23                  document.getElementById('digit-clock').innerHTML = "Current time: " + new Date;
24              }
25              setInterval(displayTime, 500);
26          </script>
27      </div>
```

Figure 10: The script block containing the setInterval function for the digital clock and canvas setup for the analog clock.

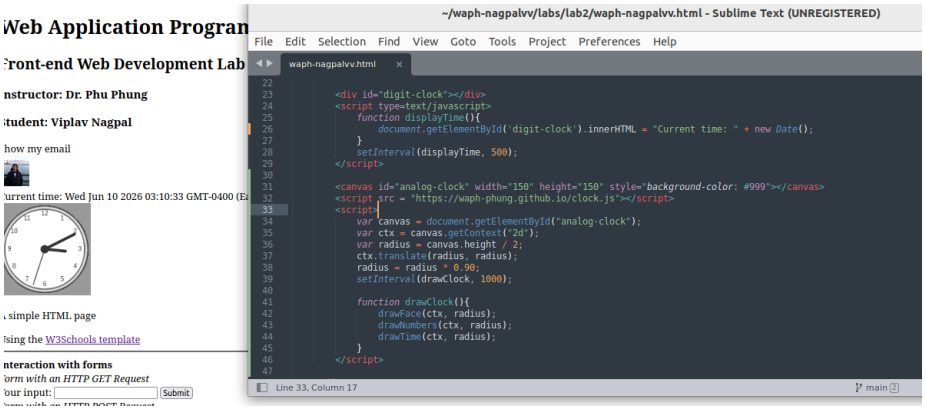

Figure 11: A split view showing the clock implementation code alongside its live rendering in the browser.

```
78
79      <hr>
80      <i>Ajax requests</i><br>
81      Your input:
82      <input name = "data"
83             onkeypress="console.log('You have presses a key')" id = "data">
84      <input type = "button" value = "submit" onclick = "getEcho()">
85      <div id = "response"></div>
86      <script>
87          function getEcho(){
88              var input = document.getElementById("data").value;
89              if(input.length == 0){
90                  return;
91              }
92
93              var xhttp = new XMLHttpRequest();
94              xhttp.onreadystatechange = function() {
95                  if(this.readyState == 4 && this.status == 200){
96                      console.log("Received data="+xhttp.responseText);
97                      document.getElementById("response").innerText = "Response from server:"+xhttp.
                            responseText;
98                  }
99              }
100             xhttp.open("GET","echo.php?data="+input, true);
101             xhttp.send();
102             document.getElementById('data').value="";
103         }
104     </script>
```

Figure 12: JavaScript code defining the getEcho function, which manually configures an XMLHttpRequest.

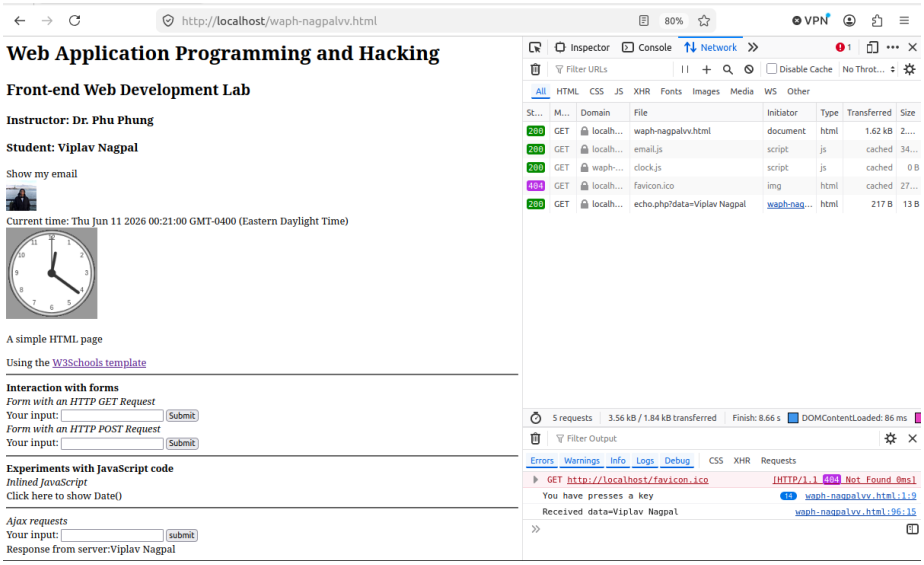

Figure 13: The webpage dynamically displaying the server's response from echo.php without a full page reload.

b. CSS

We enhanced the visual layout of the page by incorporating CSS. We linked a remote external stylesheet (`style1.css`) for the overall container and layout wrapper structure, and added internal styling classes for the buttons and response containers.

c. jQuery

To streamline our Ajax calls, we imported the jQuery 3.7.1 library. We then replicated the vanilla Ajax functionality using jQuery's simplified syntax for both GET and POST requests securely in the background.

d. Web API integration

i. Joke API (jQuery) We configured an automatic API call using jQuery's `$.get()` that triggers as soon as the JavaScript block runs. It fetches a random programming joke from `v2.jokeapi.dev` and dynamically displays it inside the response `<div>`.

ii. Agify API (Fetch) We utilized modern asynchronous JavaScript (`async/await`) alongside the `fetch()` API to predict a user's age based on their name. We updated the endpoint to `https://` to bypass CORS and mixed-content restrictions, successfully grabbing the input, querying

```
<head>
    <meta charset="utf-8">
    <title>WAPH-nagpalvv</title>

    <link rel = "stylesheet"
    href = "https://waph-phung.github.io/style1.css">
</head>
<body>
    <div id="top" class = "container wrapper">
        <h1>Web Application Programming and Hacking</h1>
        <h2>Front-end Web Development Lab</h2>
        <h3>Instructor: Dr. Phu Phung</h3>
    </div>

    <div id="menubar" class = "wrapper">
        <h3>Student: Viplav Nagpal</h3>

        <div id="email" onclick="showhideEmail()">Show my email</div>
        <script src = "email.js"></script>

        <img src="images/Headshot.jpeg" alt="My Headshot" width="50">

        <div id="digit-clock"></div>
        <script type=text/javascript>
            function displayTime(){
                document.getElementById('digit-clock').innerHTML = "Current time: " + new Date();
            }
            setInterval(displayTime, 500);
        </script>

        <canvas id="analog-clock" width="150" height="150" style="background-color: #999"></canvas>
        <script src = "https://waph-phung.github.io/clock.js"></script>
        <script>
            var canvas = document.getElementById("analog-clock");
            var ctx = canvas.getContext("2d");
```

Figure 14: HTML head section showing the link tag successfully importing the remote style1.css stylesheet.

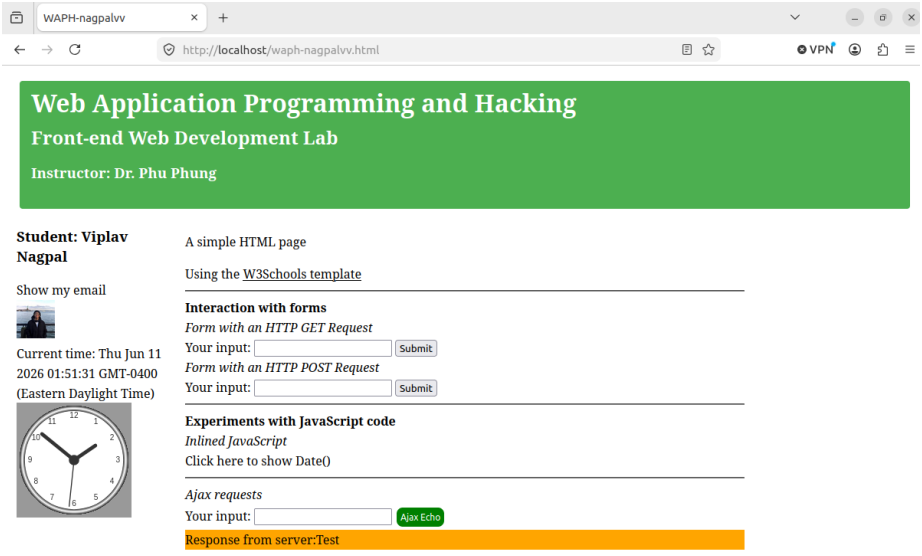

Figure 15: The browser view showing the webpage transformed by CSS into a clean, two-column layout with a green header.

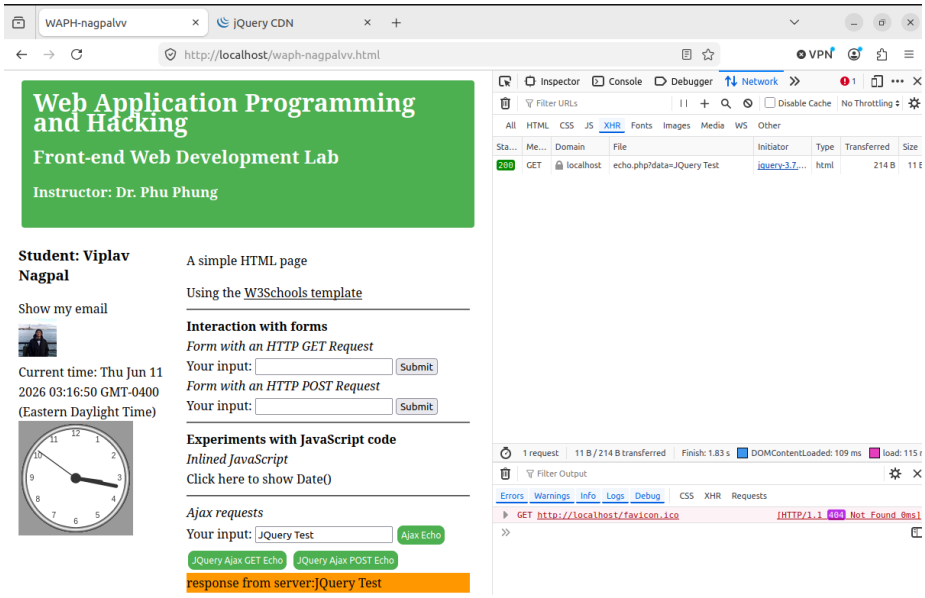

Figure 16: The browser's Network tab confirming a successful 200 OK GET request sent via jQuery.

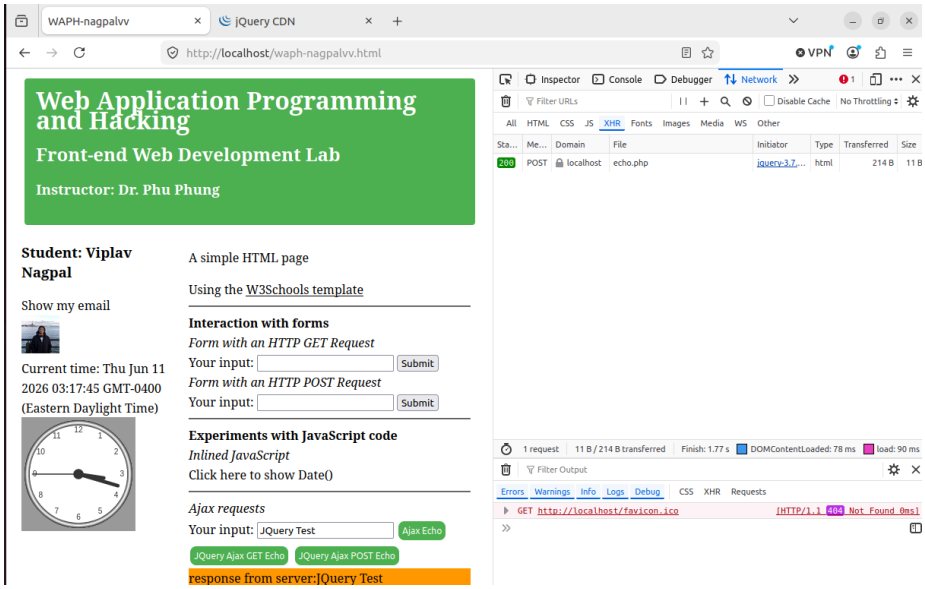

Figure 17: The browser's Network tab confirming the payload data was successfully transmitted via a jQuery POST request.

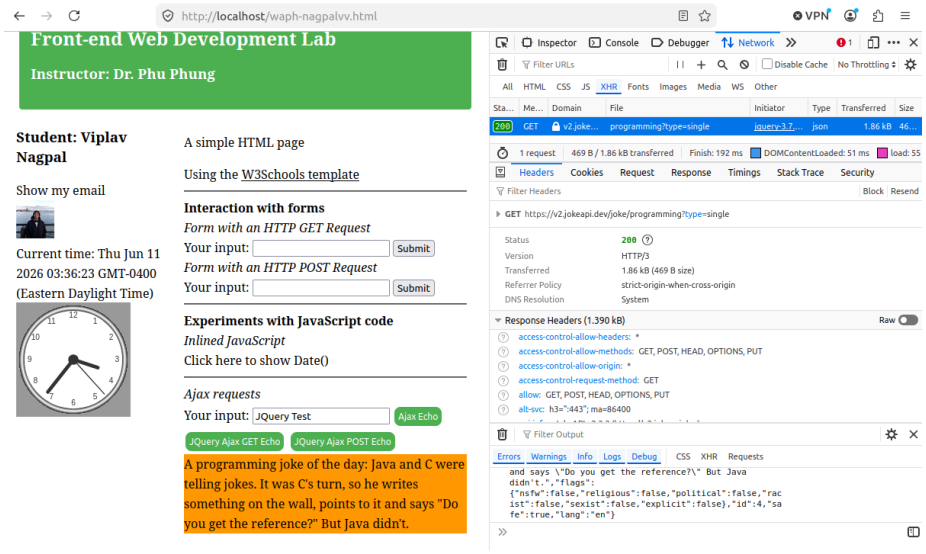

Figure 18: Network tools showing the successful automated GET request to the remote Joke API server.

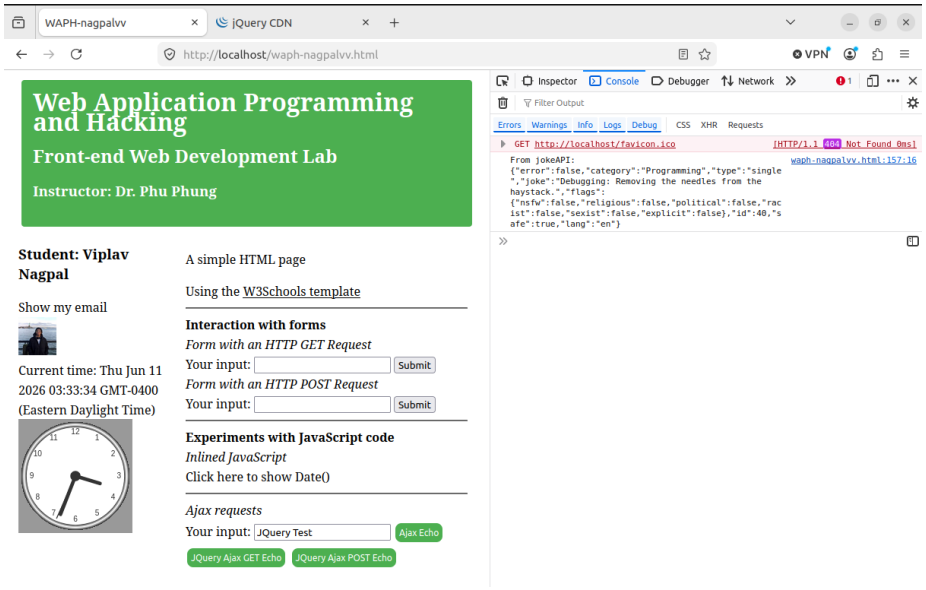

Figure 19: The developer console printing the raw, stringified JSON payload received from the Joke API.

`api.agify.io`, parsing the returned JSON, and updating the DOM.

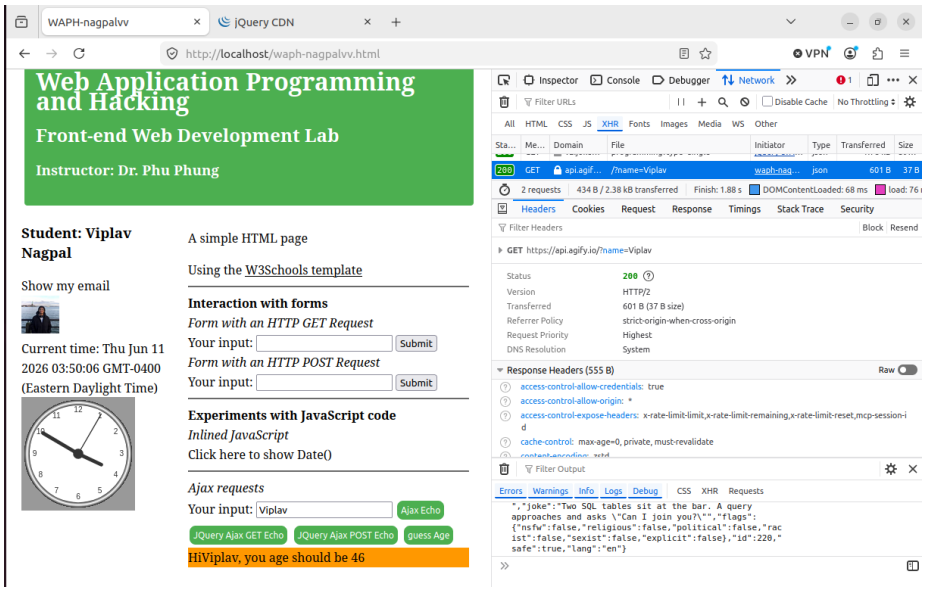

Figure 20: The final integrated webpage utilizing the modern fetch API to guess the user's age and display it in an orange notification box.